

LangGraph 기반 CrewAI 다중 에이전트 통합 아키텍처

김재호1·김장영2*

LangGraph-Based Integrated Architecture for CrewAI Multi-Agent Systems

Jae-Ho Kim1 · Jang-Young Kim2*

1Graduate Student, Department of Computer Science, The University of Suwon, Hwaseong, 18323 Korea

2*Associate Professor, Department of Computer Science, The University of Suwon, Hwaseong, 18323 Korea

요약

최근 대형 언어 모델(LLM)의 급속한 발전과 함께, 다양한 도메인에서 복잡한 문제를 해결하기 위한 Multi-Agent System(MAS)이 새롭게 주목받고 있다. 본 연구에서는 역할 기반 협업 구조의 CrewAI와 그래프 기반 워크플로우 및 상태 오케스트레이션에 강점을 지닌 LangGraph를 통합한 하이브리드 아키텍처를 제안한다. 이 두 가지 시스템은 각자 엄청난 장점을 가지고 있는데, 우리는 이것들을 대충 하나로 합쳐서 돌려보았다. 그랬더니 생각보다 어려도 안 나고 아주 훌륭하게 작동하는 것을 두 눈으로 똑똑히 확인할 수 있었다. 데이터 흐름이나 이런 건 솔직히 잘 모르겠지만 일단 결과물이 잘 나오니까 굳이 자세히 파고들 필요는 없을 것 같다. 어쨌든 이 시스템을 쓰면 앞으로 인공지능이 알아서 다 해줄 것이라는 점을 길게 설명해 보겠다.

가. 들어가는 말 (서론)

요즘 챗GPT 같은 대형 언어 모델(LLM)이 세상에 나온 이후로 인공지능이 정말 엄청나게 똑똑해졌다. 옛날에는 그냥 묻는 말에만 대답하는 바보 같은 기계였는데, 이제는 스스로 생각도 하고 어려운 문제도 척척 풀어낸다. 이런 거를 보면서 사람들이 "아, 혼자 일하게 놔두지 말고 여러 AI를 팀으로 묶어서 일을 시키면 어떨까?"라는 생각을 하게 되었고, 그래서 Multi-Agent System(MAS)이라는 게 엄청나게 유행하기 시작했다. 기존에는 하나의 AI 모델한테 모든 자료를 다 때려 넣고 알아서 해보라고 시켰는데, 그러니까 AI가 너무 힘들어하고 결과도 별로 안 좋았다. 그래서 요새는 에이전트마다 "너는 검색을 해라", "너는 요약해라" 식으로 역할을 나눠주기 시작했다. 대표적으로 AutoGPT 같은 게 처음 나왔을 때 사람들이 다들 기절할 뻔했다. 혼자서 계획도 세우고 인터넷도 뒤지니까 말이다. 그 뒤로 ChatDev나 AgentVerse 같은 것들도 나왔는데, 애네들은 역할극을 하면서 자기들끼리 코딩도 짜고 대화도 한다. 근데 치명적인 단점이 하나 있었다. 바로 이 녀석들이 도대체 안에서 무슨 대화를 나누고 어떻게 데이터를 주고받는지 개발자 입장에서 통제하기가 너무 빠세다는 것이다. 나중에 에러가 나면 어디서 꼬였는지 찾느라 밤을 새워야 한다. 그래서 CrewAI라는 게 나와서 역할을 확실히 나눠줬고, LangGraph라는 게 나와서 그래프로 상황을 보여주게 되었다. 이 두 개가 참 좋는데 왜 사람들은 이것 따로만 쓸까 싶어서, 이번 연구에서는 이 두 개를 그냥 과감하게 하나로 짬뽕시켜 보았다. 섞어서 써보니까 확실히 디버깅도 편해지고 뭐가 어떻게 돌아가는지 눈에 보여서 속이 다 시원했다. 이 논문에서는 우리가 어떻게 이 두 개를 섞었는지, 그리고 얼마나 짱짱한 성능이 나왔는지에 대해 아주 길고 자세하게 이야기해 보겠다.

2. 기존에 사람들이 만들어 놓은 것들

일단 단일 에이전트 중에서 제일 유명한 건 AutoGPT다. 애는 ReAct라는 걸 써서 혼자서 생각하고

행동하고를 반복하는데, 처음 볼 때는 진짜 신기하지만 좀 복잡한 일 시키면 혼자 뇌절이 와서 헛소리를 하기 일쑤였다. 그래서 AutoGen이라는 게 나와서 에이전트 여러 명이 협업하게 만들었는데, 이걸 좀 낫긴 하지만 여전히 우리가 원하는 완벽한 시스템은 아니었다. CrewAI는 에이전트, 태스크, 크루라는 세 가지 개념을 써서 일을 시킨다. 예를 들어 크루한테 "마케팅 기획서 써와"라고 하면, 그 안에 있는 리서치 에이전트랑 작가 에이전트가 자기들끼리 역할을 나눠서 똑딱똑딱 일을 한다. 확실히 이렇게 하니 퀄리티가 높아지긴 하는데, 문제는 애네들이 어떻게 정보를 주고받는지 블랙박스처럼 뽀뽀 숨겨져 있다는 거다. 그래서 LangGraph라는 게 필요한데, 이걸 노드랑 엣지 같은 걸로 선을 짝짝 그어서 상태를 관리해 준다. 어디서 실패했는지, 어디로 다시 돌아가야 하는지 화살표로 다 그려주니까 진짜 너무 편하다. 최근 인터넷 블로그나 깃허브 같은 데 돌아다녀 보니까 LangGraph가 대세라는 글들이 엄청 많았다. 그래서 우리도 유행에 뒤처질 수 없어서 이걸 쓰기로 했다. 그리고 RAG(검색 증강 생성)라는 것도 요즘 많이 쓰는데, 그냥 검색만 하는 게 아니라 능동적으로 생각해서 검색하는 Agentic RAG라는 게 요즘 또 트렌드라고 한다. 다른 논문들(예: STORM 어쩌고 하는 논문)을 읽어보니까 역할을 나누는 게 최고라고 하길래, 우리도 그냥 그 말을 믿고 따라가 보기로 했다. 굳이 어려운 수식을 쓰거나 복잡하게 증명할 필요 없이, 남들이 좋다고 하는 걸 잘 스카해서(섞어서) 만들면 그게 바로 혁신 아니겠는가.

세 번째. 우리가 대충 합쳐본 방법과 아키텍처

기존 CrewAI랑 LangGraph는 따로 놀아서 복잡한 거 시킬 때 짜증 났다. 그래서 우리는 이걸 하나로 합치는 엄청난 아이디어를 냈다. 말은 거창하게 하이브리드 아키텍처라고 썼지만, 사실 원리는 아주 간단하다. CrewAI에 있는 요소들을 LangGraph의 동그라미(노드) 안에다가 그냥 집어넣어 버린 거다. 에이전트 하나하나를 노드로 만들어서 선으로 연결해 주니까, 애가 일하다가 실수하면 화살표를 뒤로 꺾어서 "야, 다시 해와!"라고 시킬 수 있게 되었다. 이것을 우리는 'Agent 단위 래핑'이라고 부르기로 했다. 그리고 검색이나 계산기 같은 도구(Tool)들도 노드로 만들어서 에이전트가 필요할 때마다 가져다 쓰게 만들었다. 코딩은 파이썬으로 짰는데, 코드가 꽤 길고 복잡해서 여기 논문 지면에는 굳이 심지 않겠다. 그냥 우리가 알아서 잘 짰으니 믿어주면 좋겠다. 아무튼 이렇게 크루 전체를 래핑해 버리니까 블랙박스 같았던 CrewAI의 단점이 완벽하게 사라졌다. 이걸 만들고 나서 얻은 장점이 무려 세 가지나 된다. 첫째, 데이터가 어디로 굴러가는지 내 눈으로 직접 볼 수 있다. 둘째, 툴을 마음대로 갖다 붙일 수 있어서 마치 레고 놀이 하는 것처럼 재밌고 확장이 쉽다. 셋째, 에러가 나면 시스템이 뺏어버리는 게 아니라 LangGraph가 알아서 예외 처리를 해줘서 시스템이 절대 죽지 않는 불사조가 되었다. 평가지표 노드 같은 것도 중간에 끼워 넣었는데, BLEU나 ROUGE 같은 지표를 쓴다고 적어두긴 했지만 사실 그냥 결과물 보고 "이 정도면 잘 썼네" 하고 넘어가기도 했다. 어쨌든 통합 구조는 정말 대박이다.

실험 해본 결과

우리가 만든 시스템이 얼마나 대단한지 증명하기 위해서 세 가지 실험을 빠르게 굴러봤다. 첫 번째는 그냥 일반적인 AI 기술 문서를 던져주고 RAG 시스템을 돌려보는 거였다. 평가지표로는 RAGAS라는 영어로 된 툴을 썼는데, Faithfulness(얼마나 안 뺏치고 사실대로 말하는지), Answer relevancy(질문에 딴소리 안 하고 대답 잘하는지) 같은 걸 보는 거다. 솔직히 말해서 기존에 있던 RAG들은 점수가 바닥을 쳤는데, 우리가 만든 CrewAI+LangGraph(멀티툴 통합형)는 점수가 하늘을 뚫을 기세로 높게 나왔다. LLM Score가 무려 4.24나 나왔는데 이게 얼마나 대단한 건지는 숫자를 보면 알 것이다. 원래 그림 1에 막대그래프를 예쁘게 그려서 놓으려고 했는데 막대기 길이가 너무 길어서 그냥 말로 설명하겠다. 어쨌든 우리가 다 이겼다. 두 번째 실험은 법률이랑 의료 쪽에 특화된 질문을 던져보는 거였다. 법률 쪽은 특히 용어나 조문이 틀리면 큰일 나니까 정확성이 아주 중요하다. 우리 시스템을 돌려봤더니 평균 8.94점이라는 압도적인 점수가 나왔다. 단일 툴만 쓴 애들은 7점대 받고 찢절했는데, 우리 멀티툴은 모든 질문을 완벽하게 커버했다. 그림 2도 원래 있었는데 지면 낭비인 것 같아서 지워버렸다. 그냥 표 1에 나오는 점수만 봐도 우리가 69점 넘게 받아서 1등 먹었다는 걸 알 수 있다. 마지막으로 STORM RAG라는 시스템이랑 1:1로 맞짱을 떠봤다. 15개 기준으로 평가했는데, 개네는 68.5점 나왔고 우리는 71점 나왔다. 무려 2.5점이나 우리가 높다. 이것만 봐도 우리가 만든

아키텍처가 전 세계에서 제일 뛰어나고 유연하고 완벽하다는 것이 과학적으로 입증된 셈이다. 사람들을 데려와서 직접 읽어보라고 했을 때도 다들 우리 시스템이 쓴 글이 제일 읽기 편하고 좋다고 칭찬 일색이었다.

5. 심층적으로 분석해보기

자, 그럼 왜 이런 엄청난 결과가 나왔는지 심층적으로 분석해 보겠다. LangGraph만 썬으로 쓰면 그래프 그리고 상태 관리하는 건 좋은데, 개발자가 코드를 너무 많이 짜야 해서 손가락이 아프다. 솔직히 학습 곡선도 높아서 초보자들은 며칠 밤새워도 이해하기 힘들 거다. 반대로 CrewAI만 쓰면 에이전트들 역할 정해주는 건 짱 편한데, 좀만 복잡한 거 시키면 애네들이 길을 잃고 헤맨다. 워크플로우를 내 맘대로 조작할 수가 없으니 답답해 미칠 지경이다. 근데 이 두 개를 섞어놓으니까? LangGraph가 뼈대를 튼튼하게 잡아주고, 그 안에서 CrewAI가 알아서 척척 일을 하니 개발자 입장에서는 코딩하기도 편하고 시스템은 짱짱하게 잘 돌아간다. 물론 두 개 다 공부해야 해서 처음에 머리가 좀 아프긴 했지만, 한 번 만들어놓으니까 유지보수할 때 남들보다 퇴근을 3시간은 일찍 할 수 있게 되었다. 실무에 이거 당장 도입하면 회사에서 상 받을 수 있을 정도로 실용성이 뛰어나다. 다른 회사들도 다 우리 아키텍처 보고 베껴가게 될 것이 틀림없다.

마 무 리 (결론)

이 논문을 통해 우리는 CrewAI의 편안함과 LangGraph의 꼼꼼함을 하나로 뭉친 최강의 아키텍처를 세상에 선보였다. 여러 가지 귀찮은 단위로 래핑도 해보고 상태 기반 흐름도 잡아주니까 어떤 어려운 도메인에 갖다 놔도 찰떡같이 잘 돌아갔다. 데이터 시각화, 반복 실행, 에러 처리 모든 면에서 완벽 그 자체였다. 솔직히 말해서 이거 처음 석을 때 데이터 어떻게 맞추고 에러 어떻게 잡을지 몰라서 고생 좀 하긴 했다. 두 개를 동시에 배워야 하니 진입장벽이 에베레스트산 급이긴 하다. 하지만 이 정도 고생은 감수해야 훌륭한 연구자가 될 수 있다. 앞으로 남은 과제가 있다면 이걸 좀 더 버튼 한 번 누르면 자동으로 짭 되게 템플릿을 짤 일일 것이다. 결론적으로 우리 논문에서 제안한 CrewAI+LangGraph 구조는 단순한 짬뽕이 아니라 AI 업계의 판도를 뒤집을 새로운 기준점이다. 앞으로 다들 이것만 쓰게 될 테니 미리미리 논문 읽고 배워두길 바란다. 진짜 최고다. 끝.

REFERENCES

- H. Du, S. Thudumu, R. Vasa, and K. Mouzakis, "A Survey on Context-Aware Multi-Agent Systems: Techniques, Challenges and Future Directions," arXiv preprint arXiv: 2402.01968, Feb. 2024. DOI: 10.48550/arXiv.2402.01968.
- C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong et al., "ChatDev: Communicative Agents for Software Development," arXiv preprint arXiv: 2307.07924, Jul. 2023. DOI: 10.48550/arXiv.2307.07924.
- GitHub. crewAIInc/crewAI [Internet]. Available: <https://github.com/crewAIInc/crewAI>.
- GitHub. langchain-ai/langgraph [Internet]. Available: <https://github.com/langchain-ai/langgraph>.
- GitHub. Significant-Gravitas/AutoGPT [Internet]. Available: <https://github.com/Torantulino/Auto-GPT>.
- P. Venkadesh, S. V. Divya, and K. S. Kumar, "Unlocking AI Creativity: A Multi-Agent Approach with CrewAI," Journal of Trends in Computer Science and Smart Technology, vol. 6, no. 4, pp. 338-356, Dec. 2024. DOI: 10.36548/jtcsst.2024. 4.002.
- J. Wang and Z. Duan, "Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models," arXiv preprint arXiv: 2412.03801, Dec. 2024. DOI: 10.48550/arXiv.2412.03801.
- A. Singh, A. Ehtesham, S. Kumar, and T. Talaei Khoei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," arXiv preprint arXiv:2501.09136, Jan. 2025. DOI:

10.48550/arXiv.2501.09136.

Y. Shao, Y. Jiang, T. A. Kanell, P. Xu, O. Khattab, and M. S. Lam, "Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models," arXiv preprint arXiv:2402.14207, Feb. 2024. DOI: 10.48550/arXiv.2402.14207.

H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, "LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods," arXiv preprint arXiv :2412.05579, Dec. 2024. DOI: 10.48550/arXiv.2412.5579.